



Dompé

DOCUMENTAZIONE TECNICA

High Level Design

Architettura della soluzione di gestione degli accessi Wi-Fi ospiti sui controller Cisco Catalyst 9800



Campo	Valore
Applicativo	Guest Portal — Cisco Catalyst 9800 Guest Management Desk
Documento	HLD — High Level Design
Versione	1.0
Data di emissione	22 settembre 2026
Stato	Emesso
Destinatari	IT Infrastructure, Security, Sviluppo, Service Desk
Classificazione	Uso interno
Repository di riferimento	github.com/ACF-Dompe/Centralino
Revisione del codice	main · f427890

Indice

1	Scopo e ambito del documento.....	3
1.1	Obiettivo.....	3
1.2	Destinatari.....	3
1.3	Documenti correlati.....	3
2	Contesto	4
2.1	Il problema	4
2.2	Gli attori.....	4
2.3	Perimetro funzionale.....	4
2.4	Sedi servite	4
3	Visione d'insieme	6
3.1	Principi architetturali	6
3.2	Componenti	7
3.3	Perché nessun CORS	7
4	Flussi principali.....	8
4.1	Accesso alla console	8
4.2	Rilascio di un accesso ospite	8
4.3	Scadenza e revoca.....	8
4.4	Amministrazione	8
5	Modello di sicurezza (sintesi)	9
5.1	Autenticazione	9
5.2	Autorizzazione	9
5.3	Segreti e dati sensibili.....	9
6	Modello dati (sintesi)	10
7	Architettura di deployment	11
7.1	Ambienti.....	11
7.2	Pipeline	11
7.3	Modello consume-only.....	11
7.4	Ingress	12
8	Requisiti non funzionali	13
9	Decisioni architetturali	14
10	Rischi e deviazioni accettate.....	15
10.1	Rischi residui	15
11	Evoluzioni previste	16

1 Scopo e ambito del documento

Questo documento descrive l'architettura di alto livello del Guest Portal: quali componenti esistono, come sono collegati, quali responsabilità hanno e su quali decisioni progettuali si regge l'insieme. Non entra nel dettaglio implementativo, che è oggetto del documento LLD.

1.1 Obiettivo

Il Guest Portal è la console con cui il personale Dompé rilascia, sorveglia e revoca gli accessi Wi-Fi destinati ai visitatori delle sedi aziendali. Sostituisce l'uso diretto della interfaccia di amministrazione dei controller wireless **Cisco Catalyst 9800**, che richiedeva credenziali di rete privilegiate e non lasciava traccia di chi avesse rilasciato quale accesso.

L'applicazione persegue tre risultati: **rendere l'operazione accessibile** a personale non specialistico (reception, service desk), **mantenere il controllo** su chi può operare e su quale sede, e **lasciare una traccia** di ogni azione eseguita sui controller.

1.2 Destinatari

Ruolo	Che cosa trova in questo documento
Architetti e IT Infrastructure	Vista d'insieme, componenti, dipendenze di piattaforma, requisiti non funzionali.
Sicurezza e Compliance	Perimetro, modello di autenticazione e autorizzazione, deviazioni accettate (dettaglio nel documento SEC).
Sviluppo	Contesto architetturale entro cui si collocano le scelte di dettaglio descritte nel documento LLD.
Esercizio e Service Desk	Comprensione dei flussi e dei modi di degrado del servizio (dettaglio nei documenti FUN e OPS).

Tabella 1 — Destinatari del documento e relative sezioni di interesse.

1.3 Documenti correlati

Codice	Documento	Contenuto
HLD	High Level Design	Questo documento: architettura d'insieme.
LLD	Low Level Design	Struttura del codice, moduli, contratti interni, test.
DB	Schema del database	Tabelle, vincoli, indici, migrazioni, dati di riferimento.
FUN	Funzionamento dell'applicativo	Flussi funzionali passo per passo e comportamento dell'interfaccia.
API	API Reference	Contratto REST e WebSocket, codici di errore, matrice permessi.
SEC	Sicurezza e conformità	Modello di sicurezza, controlli, deviazioni consapevoli, stato di conformità.
OPS	Runbook operativo	Deploy, configurazione, manutenzione, troubleshooting, rollback.

Tabella 2 — Insieme documentale della soluzione.

2 Contesto

2.1 Il problema

Un visitatore che arriva in una sede Dompé ha bisogno di una connessione Wi-Fi temporanea. Sui controller Catalyst 9800 questo si traduce nella creazione di un utente locale di tipo `network-user` con una durata (`lifetime`) e nella sua rimozione a scadenza. Farlo a mano comporta tre problemi: richiede accesso amministrativo al controller, è soggetto a errore, e non lascia una traccia consultabile di chi ha rilasciato l'accesso e a chi.

2.2 Gli attori

Attore	Descrizione	Interazione con il sistema
Operatore di sede	Reception, front office, service desk.	Registra ospiti, consegna o rinvia credenziali, elimina accessi della propria sede.
Amministratore	IT Infrastructure / Network.	Profila gli utenti, configura le sedi e i controller, consulta lo storico, gestisce gli account di emergenza.
Osservatore	Ruolo di sola lettura.	Consulta la lista ospiti delle sedi concesse, senza poter modificare nulla.
Ospite	Il visitatore.	Non usa l'applicazione: riceve le credenziali via e-mail o su carta e si connette alla rete Wi-Fi.
Team di piattaforma	Chi possiede le risorse Azure.	Pre-provisiona l'infrastruttura, popola Key Vault, gestisce rete e ingress.

Tabella 3 — Attori del sistema.

2.3 Perimetro funzionale

Nel perimetro: autenticazione aziendale degli operatori, selezione della sede di lavoro, creazione e revoca degli account ospite sul controller della sede, consegna delle credenziali via e-mail o a voce, monitoraggio in tempo reale della durata residua, riconciliazione periodica con i controller, amministrazione di utenti e sedi.

Fuori perimetro: la configurazione radio e di rete dei controller, la gestione del captive portal, l'anagrafica dei visitatori in sistemi terzi, l'invio SMS (la funzione è presente a schema ma dormiente), e la creazione delle risorse Azure, che appartiene al team di piattaforma.

2.4 Sedi servite

L'applicazione è multi-sede per costruzione: ogni sede ha il proprio controller, i propri operatori abilitati e i propri ospiti. Il confine fra sedi è applicato dall'API, non soltanto dall'interfaccia.

Codice	Sede	Città	Controller	SSID
MIL	Dompé Milano HQ	Milano	172.18.106.100	Dompe Guest
AQ	Dompé L'Aquila	L'Aquila	172.18.106.101	Dompe Guest AQ

Codice	Sede	Città	Controller	SSID
NA	Dompé Napoli	Napoli	172.18.106.102	Dompe Guest NA
TIR	Dompé Tirana	Tirana (AL)	172.18.106.103	Dompe Guest TIR
SM	Dompé San Mateo	San Mateo (USA)	172.18.106.104	Dompe Guest SM

Tabella 4 — Sedi configurate come dati di riferimento (``backend/src/db/seed.ts``).

3 Visione d'insieme

La soluzione è composta da due container indipendenti — una single-page application servita da nginx e una API Node.js — pubblicati su Azure Container Apps dietro un Application Gateway con WAF. La API è l'unico componente che parla con il database, con Key Vault, con Microsoft Graph e con i controller wireless.

• Architettura logica della soluzione — Guest Portal

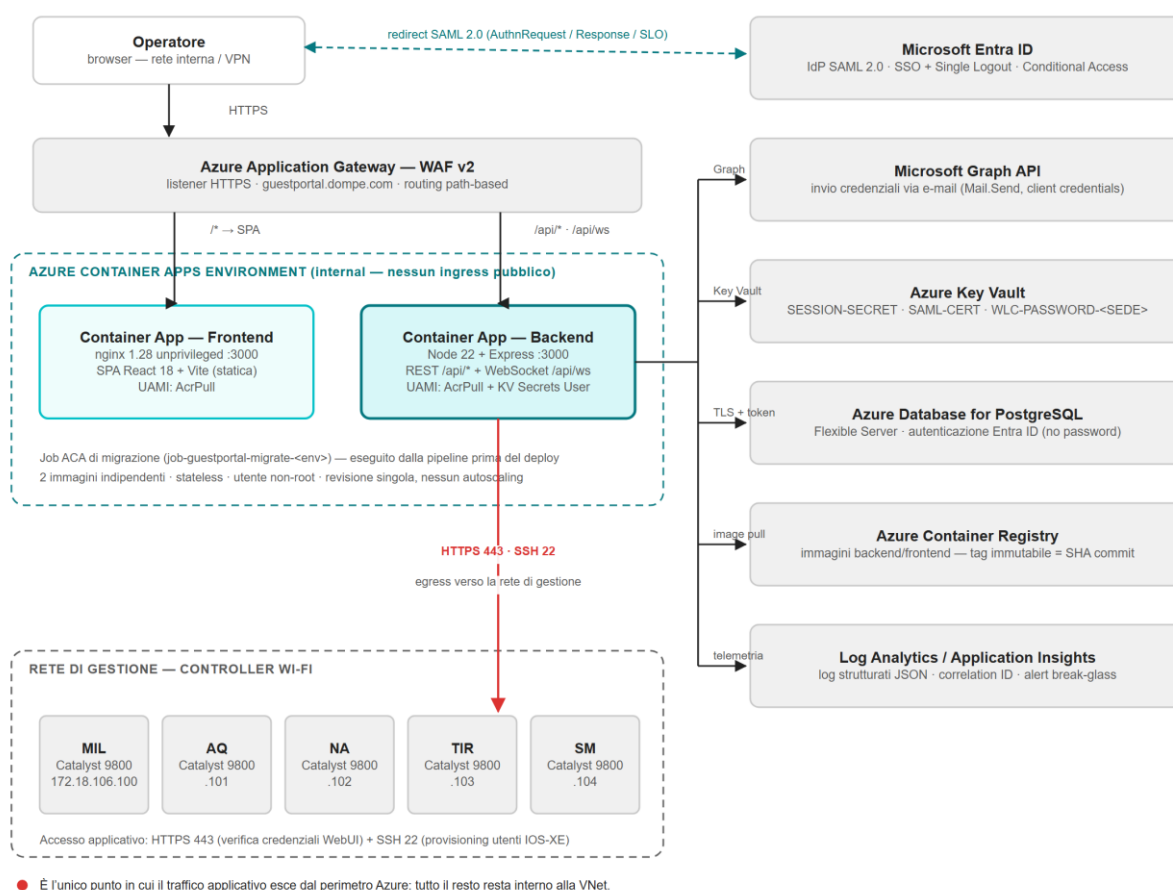


Figura 1 — Architettura logica: percorso della richiesta, servizi di piattaforma consumati e unico punto di uscita verso la rete di gestione.

3.1 Principi architetturali

- **Due immagini, nessuno stato locale.** Frontend e backend sono immagini separate, entrambe stateless e con utente non privilegiato: qualunque replica può servire qualunque richiesta.
- **Un solo perimetro di uscita.** L'unico traffico che lascia l'ambiente Azure verso la rete aziendale è quello diretto ai controller (HTTPS 443 e SSH 22). Tutto il resto resta su servizi di piattaforma.
- **Nessun segreto nel codice e nessun segreto nel database.** Ogni credenziale vive in Key Vault ed è iniettata come variabile d'ambiente; la password amministrativa del controller non è mai persistita.
- **Fail closed.** Ogni percorso di autorizzazione che non può concludersi rifiuta la richiesta: un errore nella risoluzione dei permessi produce 503, mai un ruolo di ripiego.

- **Il controller è la fonte di verità sull'accesso alla rete.** La banca dati applicativa lo rispecchia e viene riallineata periodicamente, non il contrario.
- **Modello consume-only.** L'applicazione e la sua pipeline consumano risorse pre-provisionate; non ne creano.

3.2 Componenti

Componente	Tecnologia	Responsabilità
Frontend	React 18 · Vite · Tailwind · nginx 1.28 unprivileged	Interfaccia operatore e pannello di amministrazione. Non contiene logica di autorizzazione: nasconde ciò che l'utente non può fare, ma non lo impedisce.
Backend	Node.js 22 · Express · TypeScript (ESM)	API REST sotto <code>/api</code> , canale WebSocket, autenticazione, autorizzazione, dialogo con i controller, invio e-mail, servizi di background.
Database	Azure Database for PostgreSQL Flexible Server	Ospiti, sedi, directory utenti, sessioni, account di emergenza, storico delle operazioni.
Job di migrazione	Azure Container Apps Job	Applica lo schema prima che la nuova revisione del backend entri in servizio.
Application Gateway	Azure Application Gateway WAF v2	Terminazione TLS, WAF in prevenzione, instradamento per percorso: <code>/api/*</code> al backend, tutto il resto al frontend.
Identity Provider	Microsoft Entra ID (SAML 2.0)	Autenticazione degli operatori, Conditional Access, Single Logout.
Key Vault	Azure Key Vault	Segreti applicativi e password amministrative dei controller, una per sede.
Posta	Microsoft Graph (Mail.Send)	Consegna delle credenziali all'indirizzo dell'ospite.
Controller wireless	Cisco Catalyst 9800 (IOS-XE)	Sistema di destinazione: ospita gli account ospite effettivi.
Osservabilità	Log Analytics · Application Insights	Log strutturati, correlazione delle richieste, alert sugli accessi di emergenza.

Tabella 5 — Componenti della soluzione e relative responsabilità.

3.3 Perché nessun CORS

Frontend e backend sono esposti sullo stesso hostname e distinti solo per percorso: il browser non effettua mai una richiesta cross-origin verso la API. Abilitare CORS non servirebbe a nulla e amplierebbe la superficie: è deliberatamente assente.

4 Flussi principali

I flussi sono descritti qui nella loro logica; il dettaglio passo per passo è nel documento FUN.

4.1 Accesso alla console

1. L'operatore apre l'applicazione e viene reindirizzato a Microsoft Entra ID.
2. Al primo accesso riuscito viene creata una voce nella directory applicativa, **bloccata**: l'utente è autenticato ma non può fare nulla.
3. Un amministratore assegna ruolo, stato e sedi. Il cambiamento ha effetto entro pochi secondi, senza che l'utente debba riautenticarsi.
4. L'operatore sceglie la sede su cui lavorare; la sessione viene legata a quella sede.

4.2 Rilascio di un accesso ospite

1. L'operatore compila nome, referente interno e durata (preset fino a una settimana, oppure una data di fine).
2. Il server genera username e password con un generatore crittografico; la password **esiste solo in memoria**.
3. L'ospite viene registrato in banca dati senza password, e la credenziale viene spinta sul controller della sede via SSH.
4. La password è mostrata all'operatore una sola volta e, se è stato indicato un indirizzo, inviata all'ospite via Microsoft Graph.
5. Un evento WebSocket aggiorna in tempo reale le altre postazioni della stessa sede.

4.3 Scadenza e revoca

Un servizio di background calcola ogni secondo il tempo trascorso dagli account attivi e li porta a **scaduto** quando la durata è esaurita. Ogni trenta secondi una sincronizzazione legge la configurazione dei controller e riallinea la banca dati: gli utenti trovati sul controller ma sconosciuti vengono importati, quelli spariti dal controller vengono disattivati. La revoca manuale rimuove l'utente dal controller e ne aggiorna lo stato.

4.4 Amministrazione

Il pannello di amministrazione, riservato al ruolo **admin**, copre la directory degli utenti, l'anagrafica e i parametri di connessione delle sedi con relativa diagnostica, e la gestione degli account di emergenza — limitatamente ad abilitazione, disabilitazione e sblocco. La **creazione** di un account di emergenza e la **rotazione della sua password** restano fuori dalla API, disponibili solo da riga di comando.

5 Modello di sicurezza (sintesi)

Il modello completo è descritto nel documento SEC; qui se ne riassumono i pilastri.

5.1 Autenticazione

- **SSO SAML 2.0** verso Microsoft Entra ID, con firma dell'asserzione e della risposta, verifica di `audience` e `InResponseTo`, e Single Logout.
- L'applicazione **non impone il metodo di autenticazione**: l'`AuthnRequest` non contiene `RequestedAuthnContext`, così l'accesso senza password (certificato, Windows Hello, FIDO2) resta una decisione di Conditional Access del tenant.
- **Accesso di emergenza (break-glass)** locale, disattivato per impostazione predefinita, da usare solo quando l'IdP non è disponibile.

5.2 Autorizzazione

Ruolo	Può
viewer	Consultare cruscotto e lista ospiti delle sole sedi concesse.
operator	Quanto sopra, più creare, revocare e rinviare credenziali sulle sedi concesse.
admin	Quanto sopra su tutte le sedi, più il pannello di amministrazione.

Tabella 6 — Ruoli applicativi.

L'autorizzazione è risolta **a ogni richiesta** e non è mai serializzata in sessione: una sospensione ha effetto entro il tempo di vita della cache (15 secondi), non al prossimo accesso dell'utente. Lo stesso controllo si applica all'apertura del canale WebSocket.

5.3 Segreti e dati sensibili

- Ogni segreto risiede in **Azure Key Vault** ed è iniettato come variabile d'ambiente tramite l'identità gestita del backend.
- La **password amministrativa del controller** è una per sede (`WLC-PASSWORD-<CODICE>`) e non compare in alcuna tabella né in alcuna risposta della API.
- La **password dell'ospite** non viene mai persistita: è generata in memoria, consegnata e dimenticata. Non esiste un percorso per rileggerla, solo per rigenerarla.
- Il **database** è raggiunto con un token Entra ID ottenuto dall'identità gestita: nella stringa di connessione non c'è alcuna password.

6 Modello dati (sintesi)

Il modello completo, con tipi, vincoli e indici, è nel documento DB. In sintesi ruota attorno a tre nuclei.

Nucleo	Tabelle	Ruolo
Operatività	<code>guests</code> , <code>sedi</code> , <code>sync_logs</code>	Gli ospiti, le sedi con i parametri del rispettivo controller, e lo storico delle operazioni eseguite.
Identità e permessi	<code>app_users</code> , <code>app_user_sedi</code> , <code>breakglass_users</code> , <code>session</code>	Directory applicativa popolata al primo accesso, concessioni per sede, account di emergenza, sessioni persistenti.
Supporto al protocollo	<code>saml_request_ids</code> , <code>sms_config</code> , <code>wlc_config</code>	Identificativi delle AuthnRequest in attesa, configurazione SMS dormiente, residuo legacy in via di dismissione.

Tabella 7 — Raggruppamento logico delle tabelle.

Due assenze deliberate

Nello schema **non esiste** una colonna per la password dell'ospite, e **non esiste** una colonna per la password amministrativa del controller. Sono scelte progettuali, non omissioni: la prima vive solo in memoria per il tempo di una richiesta, la seconda solo in Key Vault.

7 Architettura di deployment

7.1 Ambienti

Ambiente	Suffisso	Origine del deploy	Hostname
Sviluppo	dev	Avvio manuale del workflow	guestportal-dev.dompe.com
Collaudo	stg	Push sul ramo <code>staging</code>	guestportal-stg.dompe.com
Produzione	prod	Push sul ramo <code>main</code>	guestportal.dompe.com

Tabella 8 — Ambienti e mappatura dei rami.

La convenzione di denominazione della zona aziendale prevede che la produzione **non** abbia suffisso; ogni ambiente ha il proprio Key Vault, il proprio database e la propria applicazione registrata in Entra ID.

7.2 Pipeline

- Pipeline CI/CD — 4 stadi, modello consume-only

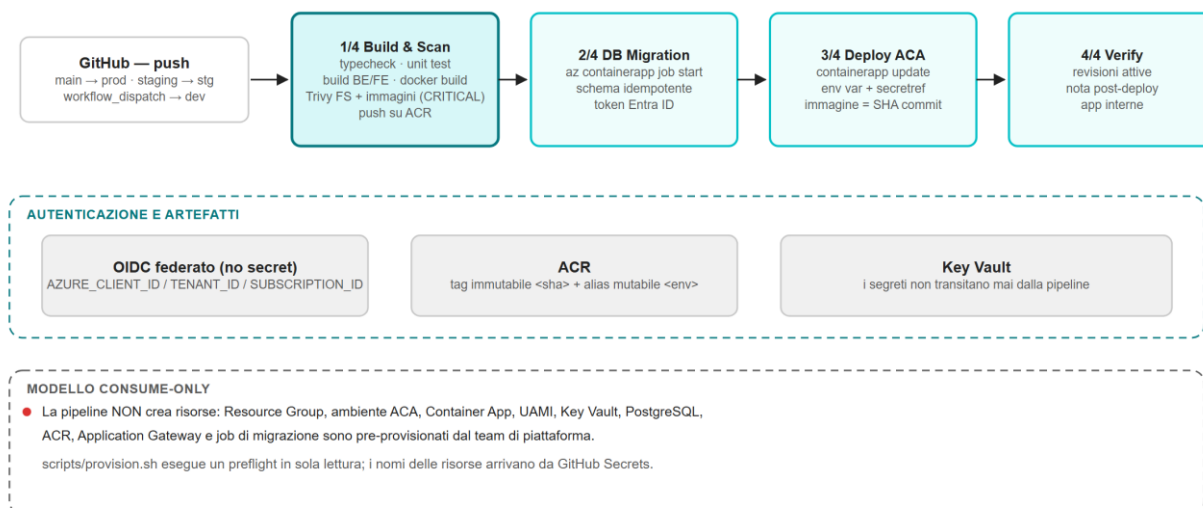


Figura 2 — Pipeline CI/CD: quattro stadi, autenticazione federata OIDC, nessuna creazione di risorse.

L'autenticazione verso Azure avviene con **identità federata OIDC**: la pipeline non custodisce alcun segreto di accesso al cloud. I segreti applicativi non transitano mai dalla pipeline, che si limita a impostare i riferimenti a Key Vault sulle variabili d'ambiente della Container App.

7.3 Modello consume-only

Resource Group, ambiente Container Apps, le due Container App, le identità gestite, Key Vault, PostgreSQL, Container Registry, Application Gateway e il job di migrazione sono **pre-provisionati dal team di piattaforma**. La pipeline esegue soltanto aggiornamenti; lo script `scripts/provision.sh` effettua una verifica in sola lettura della presenza delle risorse attese.

7.4 Ingress

L'ambiente Container Apps è **internal**: non possiede un indirizzo pubblico. Le due applicazioni sono create con ingress [external](#), che in Container Apps significa «raggiungibile dall'intera rete virtuale», non «esposto su Internet»: è la condizione necessaria perché l'Application Gateway, che vive fuori dall'ambiente, possa raggiungerle. La conformità «nessun ingress pubblico» è garantita dall'ambiente, non dal singolo flag applicativo.

8 Requisiti non funzionali

Ambito	Scelta architetturale	Conseguenza operativa
Disponibilità	Componenti stateless, sessioni su PostgreSQL, revisione singola senza autoscaling.	Il riavvio di una replica non disconnette gli operatori. La disponibilità dipende da quella del database e dell'Application Gateway.
Prestazioni	Cruscotto aggiornato da eventi WebSocket con polling di sicurezza a 30 s; timer lato server a 1 s.	Il carico verso la API è dominato dalla sincronizzazione periodica, non dai client.
Scalabilità	Il collo di bottiglia è il numero di sedi da sincronizzare, non il numero di operatori.	La sincronizzazione è sequenziale per sede ogni 30 s: con cinque sedi il margine è ampio.
Sicurezza	Vedi §5 e documento SEC.	WAF in prevenzione, nessun ingress pubblico, segreti solo in Key Vault.
Osservabilità	Log strutturati JSON con identificativo di correlazione su ogni richiesta; sonde <code>/api/healthz</code> e <code>/api/readyz</code> .	Ogni richiesta è ricostruibile in Log Analytics; gli accessi di emergenza sono registrati a livello <code>warn</code> per l'alerting.
Manutenibilità	Migrazioni idempotenti eseguite da un job dedicato prima del deploy.	Un rollback dell'immagine non richiede un rollback dello schema, purché le migrazioni restino additive.
Continuità	Persistenza affidata al backup gestito di PostgreSQL Flexible Server.	RPO e RTO coincidono con quelli definiti dal team di piattaforma per il servizio gestito.
Internazionalizzazione	Interfaccia in italiano e inglese, commutabile senza ricaricare la pagina.	Le sedi estere (Tirana, San Mateo) usano la stessa installazione.

Tabella 9 — *Requisiti non funzionali e relative scelte.*

9 Decisioni architetturali

Le decisioni seguenti hanno effetti che attraversano più componenti; sono riportate con la motivazione perché una loro revisione futura possa partire dal problema originario e non dalla sola soluzione.

#	Decisione	Motivazione
AD-01	I parametri del controller vivono sul record di sede, non su una tabella separata collegata da entrambi i lati.	Il doppio collegamento poteva divergere, e ogni lettura portava con sé un ripiego. Una riga per sede elimina la classe di errore.
AD-02	La sede su cui si opera è tenuta in sessione e non accettata dal client.	Quando arrivava dalla query string, chiunque poteva leggere gli ospiti di un'altra sede modificando l'URL.
AD-03	L'autorizzazione è risolta a ogni richiesta, mai letta dalla sessione.	Un ruolo memorizzato in sessione avrebbe tenuto in vita i permessi di un utente sospeso fino alla scadenza del cookie (24 ore).
AD-04	La password dell'ospite non viene persistita.	Riduce l'impatto di una compromissione della banca dati; la riconsegna avviene per rigenerazione, non per riletture.
AD-05	La password amministrativa del controller è una per sede e risiede solo in Key Vault.	Evita che la schermata di accesso diventi un modo per riconfigurare un controller, e limita il raggio di una compromissione a una sola sede.
AD-06	Gli identificativi delle AuthnRequest SAML sono persistiti su PostgreSQL.	La cache in memoria di <code>node-saml</code> non sopravvive a un riavvio fra il redirect e la risposta dell'IdP, e non funziona affatto con più repliche.
AD-07	La sessione è persistita su PostgreSQL con un pool costruito dall'applicazione.	Il pool interno della libreria non usa TLS e non dispone del token Entra: contro Azure PostgreSQL non si connette.
AD-08	L'AuthnRequest non contiene <code>RequestedAuthnContext</code> .	Il valore predefinito della libreria vincolava l'accesso a «password»: ogni accesso passwordless falliva con AADSTS75011. Il metodo di autenticazione è una decisione del tenant.
AD-09	La convenzione che concede il ruolo di amministratore si valuta sullo UPN, non sull'indirizzo di posta.	Gli account amministrativi non hanno casella: valutata sulla posta, la regola mancava esattamente gli account per cui esiste.
AD-10	Creazione e rotazione degli account di emergenza restano fuori dalla API.	Una sessione amministrativa compromessa non deve poter creare una credenziale che aggira Entra ID.
AD-11	Il controller è la fonte di verità sull'accesso alla rete.	In caso di divergenza vince ciò che consente o nega effettivamente l'accesso, non ciò che è registrato in banca dati.

Tabella 10 — Registro delle decisioni architetturali.

10 Rischi e deviazioni accettate

Tre deviazioni dai requisiti di sicurezza di riferimento sono state assunte consapevolmente. Il dettaglio, con i controlli compensativi e il rischio residuo, è nel documento SEC.

#	Deviazione	Controlli compensativi
D1	L'accesso di emergenza non prevede un secondo fattore.	Disattivato per impostazione predefinita, lista di indirizzi consentiti, blocco dell'account e limitazione per indirizzo di origine, sessione più breve, registrazione di ogni tentativo a livello <code>warn</code> per l'alerting, account creabili solo da riga di comando.
D2	La verifica della chiave host SSH verso i controller è disattivata.	Percorso di rete interno e gestito; l'applicazione registra una segnalazione alla prima connessione non verificata perché lo stato sia visibile.
D3	La verifica del certificato TLS dei controller è disattivata in produzione.	I Catalyst 9800 presentano un certificato autofirmato; la connessione resta cifrata e confinata alla rete di gestione. La rimozione della deviazione richiede l'installazione di certificati attendibili sui controller.

Tabella 11 — Deviazioni consapevoli e controlli compensativi.

10.1 Rischi residui

- **Dipendenza da una sola istanza di database.** Un'indisponibilità di PostgreSQL blocca autenticazione e operatività: le sessioni vivono lì.
- **Riconciliazione a finestra.** Una modifica eseguita direttamente sul controller è visibile in console entro trenta secondi, non immediatamente.
- **Consegna delle credenziali via posta.** L'invio è considerato riuscito se Microsoft Graph accetta il messaggio; la consegna effettiva alla casella dell'ospite non è tracciata.

11 Evoluzioni previste

Tema	Stato attuale	Direzione
Verifica della chiave host SSH	Disattivata: esiste un solo valore atteso a fronte di cinque controller.	Introdurre una chiave attesa per sede (<code>WLC_SSH_HOST_KEY_<CODICE></code>) e attivare la verifica.
Certificati dei controller	Autofirmati.	Installare certificati attendibili e riportare a <code>true</code> la verifica TLS in produzione.
Tabella <code>wlc_config</code>	Ancora presente e scritta per compatibilità.	Rimuoverla una volta esaurita la finestra di convivenza con le revisioni precedenti.
Canale SMS	Schema e configurazione presenti, funzione dormiente.	Attivare o rimuovere: allo stato attuale è superficie inutilizzata.
Sincronizzazione	Sequenziale per sede ogni trenta secondi.	Passare a un'esecuzione parallela con limite di concorrenza se il numero di sedi cresce.
Endpoint deprecati	<code>/api/config/wlc</code> e <code>/api/health</code> mantenuti per compatibilità.	Rimuoverli dopo il ritiro delle revisioni che li usano.

Tabella 12 — Evoluzioni identificate.